University of Engineering & Management, Kolkata
Institute of Engineering and Management, Newtown Campus
Department of Computer Applications

ASSIGNMENT SUBMISSION — TOP SHEET

Technical Class Assessment — April 2026
Database Management Systems & Java with JDBC | MCA Programme

STUDENT DETAILS

Full Name	Chandan Nayak		
Roll Number	12025006015024	**Registration No.**	304202500600024
Section	A	**Semester**	2
Programme	MCA	**Academic Year**	2025-2026
Email ID	chandan.nayak2025@iem.edu.in		

ASSIGNMENT DETAILS

Subject	Java-DBMS Assignment	**Set Number**	5
Problem Title	The Voting Booth		
Language Used	Core Java & MySQL	**Database Used**	MySQL
Submission Date	01st April 2026	**Total Pages**	12

SUBMISSION CHECKLIST

#	Item	Included (✓ / ✗)	Pages / Remarks
1	Printed Java Source Code (DBConnection.java)	✓	
2	Printed Java Source Code (Main.java)	✓	
3	Printed SQL Schema File (schema.sql)	✓	
4	Screenshot(s) of Program Execution (printed)	✓	
5	Problem Statement (this top sheet attached)	✓	

DECLARATION

I hereby declare that this assignment is my own original work. I have not copied from any other student or used any unauthorised sources. I understand that plagiarism will result in zero marks and disciplinary action.

Student Signature & Date

Faculty Signature & Date

TECHNICAL CLASS ASSESSMENT — April – 2026

Degree: MCA	Stream: MCA
Due Date: _____	**Set No: 5**

🗳️ The Voting Booth

[Domain: Civic / Events]

Language	Core Java (Console Application)
Database	MySQL
Connectivity	JDBC (Java Database Connectivity)

The Story

> *"We lost the result sheet last year. Let the system count — not us."*

The students' union election at a Kolkata college is held every year. Candidates register, students vote, and the counting is done manually on paper slips — a process prone to errors, disputes, and recounting delays.

Your task is to build an Electronic Voting System using Java and MySQL.

Your Task

Build a Student Election Voting System as a Java console application using JDBC and MySQL.

Your system must allow:

- Register a candidate for a specific post (e.g., President, Secretary)
- Cast a vote for a candidate (each voter ID can vote once per post)
- Prevent duplicate votes: check before inserting
- Display results — candidate-wise vote count per post, sorted highest first

Database Schema

Create the following table(s) in MySQL:

```
CREATE TABLE candidates (
    cand_id  INT PRIMARY KEY AUTO_INCREMENT,
    name     VARCHAR(100),
    post     VARCHAR(50)
);

CREATE TABLE votes (
    vote_id   INT PRIMARY KEY AUTO_INCREMENT,
```

```
    voter_id  VARCHAR(20),
    cand_id   INT,
    post      VARCHAR(50),
    voted_at  DATETIME DEFAULT NOW(),
    FOREIGN KEY (cand_id) REFERENCES candidates(cand_id)
);
```

Required Features

Feature	Implementation Hint
Register Candidate	INSERT into candidates with name and post
Cast Vote	Check if voter_id already voted for this post; if not, INSERT
Prevent Duplicate	SELECT COUNT(*) WHERE voter_id=? AND post=? before insert
Show Results	SELECT name, COUNT(*) GROUP BY cand_id ORDER BY count DESC

Constraints

- Use Core Java only — no Spring, Hibernate, or web frameworks
- Connect to MySQL using JDBC (no JPA / ORM)
- All operations must be menu-driven via console
- No GUI required — Scanner-based input is sufficient
- Handle SQL exceptions with try-catch blocks
- Use PreparedStatement for all SQL queries

Sample Interaction

```
===== Voting System =====
1. Register Candidate
2. Cast Vote
3. View Results
4. Exit
Enter choice: 2
Enter Voter ID: S2024101  Post: President
Candidates: 1. Arjun Das  2. Priya Sen
Your vote: 1
Vote recorded successfully.
```

Submission Checklist

- DBConnection.java (JDBC connection class)
- Main.java (menu-driven main program)
- SQL file: schema.sql (CREATE TABLE statements)
- screenshot.png (terminal output showing at least 2 features working)

> *“A database is not just data. It is the memory of your application. Make it reliable.”*

main.java

```java
import java.sql.*;
import java.util.Scanner;

public class Main {

   static void registerCandidate(String name, String post) {
       try {
           Connection con = DBConnection.getConnection();
           String query = "INSERT INTO candidates(name, post) VALUES (?, ?)";

           PreparedStatement ps = con.prepareStatement(query);
           ps.setString(1, name);
           ps.setString(2, post);

           ps.executeUpdate();
           System.out.println("Candidate Registered Successfully!");

       } catch (Exception e) {
           System.out.println(e);
       }
   }

   static void showCandidatesByPost(String post) {
       try {
           Connection con = DBConnection.getConnection();
           String query = "SELECT * FROM candidates WHERE post=?";

           PreparedStatement ps = con.prepareStatement(query);
           ps.setString(1, post);

           ResultSet rs = ps.executeQuery();

           System.out.println("\nCandidates for " + post + ":");
           while (rs.next()) {
               System.out.println(rs.getInt("cand_id") + ". " + rs.getString("name"));
           }

       } catch (Exception e) {
           System.out.println(e);
       }
   }

   static void vote(String voterId, String post, int candId) {
       try {
           Connection con = DBConnection.getConnection();

           // Check duplicate vote
           String checkQuery = "SELECT COUNT(*) FROM votes WHERE voter_id=? AND post=?";
           PreparedStatement check = con.prepareStatement(checkQuery);
           check.setString(1, voterId);
```

```
        check.setString(2, post);

        ResultSet rs = check.executeQuery();
        rs.next();

        if (rs.getInt(1) > 0) {
            System.out.println("You have already voted for this post!");
            return;
        }

        // Insert vote
        String insertQuery = "INSERT INTO votes(voter_id, cand_id, post) VALUES (?, ?, ?)";
        PreparedStatement ps = con.prepareStatement(insertQuery);
        ps.setString(1, voterId);
        ps.setInt(2, candId);
        ps.setString(3, post);

        ps.executeUpdate();
        System.out.println("Vote Recorded Successfully!");

    } catch (Exception e) {
        System.out.println(e);
    }
}

static void showResults() {
    try {
        Connection con = DBConnection.getConnection();

        String query = "SELECT c.name, c.post, COUNT(v.vote_id) AS total_votes " +
                       "FROM candidates c LEFT JOIN votes v ON c.cand_id = v.cand_id " +
                       "GROUP BY c.cand_id ORDER BY total_votes DESC";

        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery(query);

        System.out.println("\n--- Results ---");
        while (rs.next()) {
            System.out.println(
                rs.getString("post") + " | " +
                rs.getString("name") + " -> " +
                rs.getInt("total_votes") + " votes"
            );
        }

    } catch (Exception e) {
        System.out.println(e);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
```

```java
while (true) {
    System.out.println("\n===== Voting System =====");
    System.out.println("1. Register Candidate");
    System.out.println("2. Cast Vote");
    System.out.println("3. View Results");
    System.out.println("4. Exit");

    System.out.print("Enter choice: ");
    int choice = sc.nextInt();
    sc.nextLine(); // clear buffer

    switch (choice) {

        case 1:
            System.out.print("Enter Candidate Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Post: ");
            String post = sc.nextLine();

            registerCandidate(name, post);
            break;

        case 2:
            System.out.print("Enter Voter ID: ");
            String voterId = sc.nextLine();

            System.out.print("Enter Post: ");
            String votePost = sc.nextLine();

            showCandidatesByPost(votePost);

            System.out.print("Enter Candidate ID: ");
            int candId = sc.nextInt();
            sc.nextLine();

            vote(voterId, votePost, candId);
            break;

        case 3:
            showResults();
            break;

        case 4:
            System.out.println("Exiting...");
            System.exit(0);

        default:
            System.out.println("Invalid choice!");
    }
}
```

```
    }
}

                    rs.getString("name") + " → " + rs.getInt("votes") + " votes"
                );
            }

        } catch (Exception e) {
            System.out.println(e);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.println("\n1. Show Candidates");
            System.out.println("2. Vote");
            System.out.println("3. Show Results");
            System.out.println("4. Exit");

            System.out.print("Enter choice: ");
            int choice = sc.nextInt();

            switch (choice) {
                case 1:
                    showCandidates();
                    break;

                case 2:
                    showCandidates();
                    System.out.print("Enter Candidate ID: ");
                    int id = sc.nextInt();
                    vote(id);
                    break;

                case 3:
                    showResults();
                    break;

                case 4:
                    System.exit(0);

                default:
                    System.out.println("Invalid choice!");
            }
        }
    }
}
```

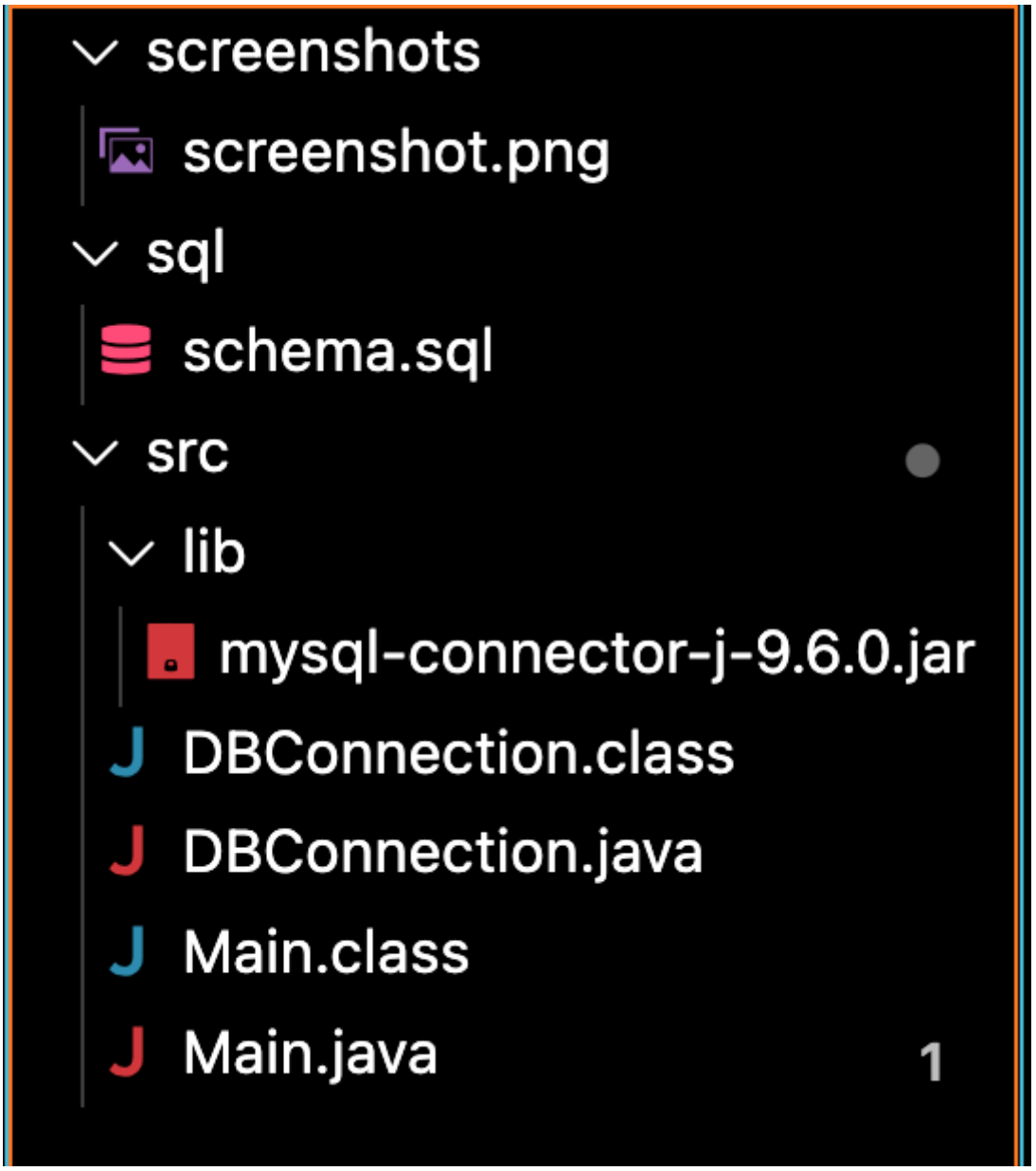
DBConnection.java

```
import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {

   static final String URL =
"jdbc:mysql://127.0.0.1:3306/voting_system1?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
";
   static final String USER = "root";
   static final String PASS = "";

   public static Connection getConnection() {
       try {
           return DriverManager.getConnection(URL, USER, PASS);
       } catch (Exception e) {
           System.out.println("Database Connection Failed!");
           e.printStackTrace();
           return null;
       }
   }
}
```

schema.sql

```sql
-- Create Database
CREATE DATABASE voting_system1;

-- Use Database
USE voting_system1;

-- Create Candidates Table
CREATE TABLE candidates (
    cand_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    post VARCHAR(50)
);

-- Create Votes Table
CREATE TABLE votes (
    vote_id INT PRIMARY KEY AUTO_INCREMENT,
    voter_id VARCHAR(20),
    cand_id INT,
    post VARCHAR(50),
    voted_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (cand_id) REFERENCES candidates(cand_id)
);
```

screenshot

```
chandannayak@Chandans-MacBook-Air src %  /usr/bin/env /Users/chandannayak/Library/Java/JavaVirtualMachines/openjdk-24.0.1/Contents/Home/bin/java -agentlib:jdwp=t
ransport=dt_socket,server=n,suspend=y,address=localhost:51533 @/var/folders/1x/7_2z1cg52ld7nlf2r7yx8qyw0000gn/T/cp_49cj8ubyivps1us3nhtfr2ryz.argfile Main

===== Voting System =====
1. Register Candidate
2. Cast Vote
3. View Results
4. Exit
Enter choice: 1
Enter Candidate Name: Chandan Nayak
Enter Post: CR
Candidate Registered Successfully!

===== Voting System =====
1. Register Candidate
2. Cast Vote
3. View Results
4. Exit
Enter choice: 2
Enter Voter ID: CR
Enter Post: CR

Candidates for CR:
1. Chandan Nayak
Enter Candidate ID: 1
Vote Recorded Successfully!

===== Voting System =====
1. Register Candidate
2. Cast Vote
3. View Results
4. Exit
Enter choice: 3

--- Results ---
CR | Chandan Nayak -> 1 votes

===== Voting System =====
1. Register Candidate
2. Cast Vote
3. View Results
4. Exit
Enter choice: 4
Exiting...
chandannayak@Chandans-MacBook-Air src %
```

Folder Structure